Enterprise Retail Analytics Platform on Azure

Technical Design and Implementation Document

Prepared By

Vuppu Reddy Kalyan

Project Domain

Data Engineering

June 2026

I am **Vuppu Reddy Kalyan**, a Bachelor of Engineering graduate in **Computer Science and Engineering (Artificial Intelligence & Machine Learning)** from **Vel Tech High Tech Dr. Rangarajan Dr. Sakunthala Engineering College**.

I have a strong interest in **Data Engineering, Cloud Computing, and Data Analytics**. Throughout my academic journey, I have built practical knowledge in **Python, SQL, PySpark, Azure Data Factory, Azure Databricks, Azure Data Lake Storage Gen2, Azure SQL Database, Delta Lake, and Power BI**. I enjoy learning modern technologies and applying them to solve real-world data processing challenges.

The **Enterprise Retail Analytics Platform on Azure** presented in this report is a significant part of my learning journey. Through this project, I designed and implemented a metadata-driven data engineering solution that automates data ingestion, validation, transformation, and publishing using Microsoft Azure services. The project follows the Medallion Architecture and demonstrates the use of reusable processing frameworks, incremental data loading, data quality validation, Slowly Changing Dimension (SCD) implementation, and business intelligence reporting.

Working on this project has strengthened my understanding of cloud-based data engineering, ETL pipeline development, distributed data processing, and scalable data platform design. It has also improved my problem-solving, analytical thinking, and technical implementation skills.

I am committed to continuously expanding my knowledge and look forward to contributing to innovative data engineering projects while growing as a technology professional.

Contact Information

Name: Vuppu Reddy Kalyan

Degree: B.E. Computer Science and Engineering (Artificial Intelligence & Machine Learning)

College: Vel Tech High Tech Dr. Rangarajan Dr. Sakunthala Engineering College

Email: vuppureddykalyan@gmail.com

GitHub: https://github.com/Kalyan31104

LinkedIn: https://www.linkedin.com/in/vuppureddykalyan/

TABLE OF CONTENTS

1. Objective

The primary objective of this project is to design and implement an end-to-end Azure Data Engineering solution for a retail analytics platform using Microsoft Azure cloud services. The solution integrates data from multiple heterogeneous sources, including CSV files, Azure SQL Database, and REST API-based JSON data, into a centralized Azure Data Lake Storage Gen2 environment for efficient storage, processing, and analytics.

---

2. Business Problem

A retail company receives data from multiple source systems, including daily sales transactions (Orders CSV), weekly product master files (Products CSV), customer master data from Azure SQL Database, and exchange rates from a REST API. Since the data is generated from different sources and in different formats, it becomes difficult to manage, process, and analyze efficiently.

The company requires a centralized data platform to integrate data from all source systems, ensure data quality, and provide accurate and consistent information for reporting and business decision-making. Manual data processing is time-consuming and increases the risk of errors, making it challenging to generate reliable business insights.

---

3. High Level Design (HLD)

3.1 Purpose

The High Level Design (HLD) provides an overall architectural view of the Enterprise Retail Analytics Platform. It illustrates the major Azure services involved in the solution and the high-level data flow from the source systems to the reporting layer. The HLD is intended to provide stakeholders with a clear understanding of the system architecture without exposing implementation-level details.

3.2 High Level Architecture

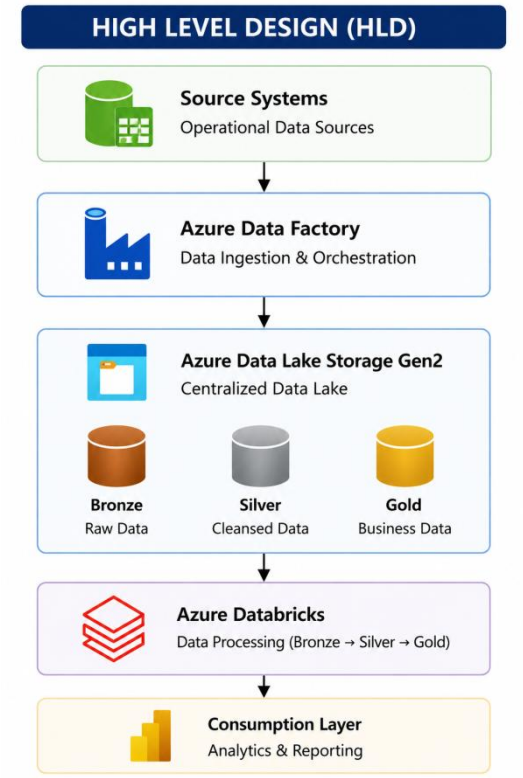


Figure 3.1: High Level Architecture of Enterprise Retail Analytics Platform

3.2 Architecture Overview

The solution follows a cloud-native architecture built on Microsoft Azure services to support the complete lifecycle of retail data processing. Data is collected from multiple operational source systems, including Orders CSV files, Products CSV files, Customer data stored in Azure SQL Database, and Exchange Rates obtained from a REST API.

Azure Data Factory (ADF) serves as the orchestration layer by extracting data from the configured source systems and coordinating the end-to-end execution of the data pipeline. The ingested data is initially stored in Azure Data Lake Storage Gen2 (ADLS Gen2), which acts as the centralized storage repository for both raw and processed datasets.

Azure Databricks provides the distributed processing engine for the solution. Using PySpark notebooks, the framework processes the ingested data through the Bronze, Silver, and Gold layers of the Medallion Architecture. The processed Gold layer datasets are published to Azure SQL Database, which serves as the reporting database for downstream analytical applications. Power BI connects to Azure SQL Database to generate interactive dashboards and business reports.

4. Low Level Design (LLD)

4.1 Purpose

The Low Level Design (LLD) provides the detailed implementation of the Enterprise Retail Analytics Platform. It describes the configuration and interaction of each Azure component, the processing logic implemented within the Medallion Architecture, and the metadata-driven framework used to automate data ingestion and transformation.

Unlike the High Level Design, the LLD focuses on implementation details, processing sequence, and component-level responsibilities.

4.2 Low Level Architecture

Figure 4.1: Low Level Architecture of Enterprise Retail Analytics Platform

4.3 Implementation Overview

The implementation follows a metadata-driven architecture in which Azure Data Factory orchestrates the complete data pipeline while Azure Databricks performs data processing using reusable PySpark notebooks. All dataset-specific configurations, including source information, validation rules, schemas, load strategies, and target tables, are maintained in a centralized metadata configuration file. During execution, the framework dynamically reads this metadata and applies the appropriate processing logic without requiring dataset-specific code.

The implementation begins by extracting data from multiple source systems and storing it in the Raw zone of Azure Data Lake Storage Gen2. The Bronze notebook ingests the raw data into Delta tables while preserving the original source records. The Silver notebook reads the Bronze data and applies schema standardization, data quality validations, duplicate removal, business rule validation, and rejected record handling based on the metadata configuration. Finally, the Gold notebook transforms the validated data into business-ready fact and dimension tables, implements Slowly Changing Dimension (SCD) Type 2 for customer data, and publishes the curated datasets for reporting.

4.4 Component Interaction

The Azure Data Factory pipeline acts as the orchestration layer and coordinates the execution of the complete data processing workflow. It invokes the Databricks notebooks by passing runtime parameters such as the dataset name, metadata configuration path, and pipeline run identifier.

Azure Databricks retrieves the required dataset configuration from the metadata file and executes the processing logic dynamically. Each notebook performs a dedicated responsibility within the Medallion Architecture, ensuring a clear separation between data ingestion, validation, and business transformation. The processed data is stored in Azure Data Lake Storage Gen2 as Delta tables, while audit information and rejected records are maintained separately to support monitoring and data quality management.

After successful Gold layer processing, the curated datasets are published to Azure SQL Database, where they become available for business reporting through Power BI dashboards.

4.5 Design Characteristics

The implementation has been designed with the following characteristics:

- **Metadata-Driven Processing:** Dataset-specific configurations are maintained externally, enabling dynamic execution without modifying notebook code.
- **Reusable Framework:** Generic Azure Data Factory pipelines and Databricks notebooks are reused for multiple datasets through parameterization.
- **Incremental Processing:** Watermark-based incremental loading minimizes unnecessary data movement and improves pipeline performance.
- **Data Quality Management:** Centralized validation rules, business rule enforcement, duplicate removal, and rejected record handling ensure high-quality datasets.

- **Historical Data Management:** Slowly Changing Dimension (SCD) Type 2 preserves historical customer information while maintaining current records for reporting.
- **Audit and Monitoring:** Audit logging captures execution details and supports end-to-end traceability across the complete data pipeline.

---

5. Components of Low Level Design

5.1 Overview

The Enterprise Retail Analytics Platform is built using multiple Azure services that work together to implement a scalable and metadata-driven data engineering solution. Each component is responsible for a specific stage of the data lifecycle, including data ingestion, orchestration, storage, processing, publishing, and reporting.

The integration of these components enables the platform to process data from multiple heterogeneous sources while maintaining data quality, scalability, and operational reliability.

5.2 Source Systems

The source systems represent the origin of the business data processed by the platform. The project integrates transactional, master, and reference data from multiple sources, including Orders and Products CSV files, Customer data from Azure SQL Database, and Exchange Rates JSON

Since each source provides data in different formats and frequencies, the solution implements a metadata-driven ingestion framework that standardizes the processing of all datasets while preserving the characteristics of the original source data.

5.3 Azure Data Factory (ADF)

Azure Data Factory serves as the orchestration layer of the solution. It is responsible for coordinating the complete execution of the data pipeline by extracting data from the configured source systems and initiating the downstream processing activities.

The implementation uses parameterized pipelines that dynamically pass runtime parameters such as dataset name, metadata configuration path, and pipeline run identifier to Azure Databricks notebooks. This enables the same pipeline to process multiple datasets without requiring separate implementations.

ADF also manages execution sequencing, dependency handling, and pipeline monitoring, ensuring that each stage of the data engineering workflow executes in the correct order.

5.4 Azure Data Lake Storage Gen2 (ADLS Gen2)

Azure Data Lake Storage Gen2 serves as the centralized storage repository for the entire platform. It stores raw source data, intermediate datasets, curated analytical data, audit logs, and rejected records generated during processing.

The storage structure follows the Medallion Architecture, separating the data into Bronze, Silver, Gold, and Audit layers. This layered organization simplifies data management, improves traceability, and enables independent processing at each stage of the data pipeline.

5.5 Azure Databricks

Azure Databricks acts as the data processing engine of the platform. Generic PySpark notebooks are used to ingest, validate, transform, and prepare datasets based on the metadata configuration.

The implementation separates processing into Bronze, Silver, and Gold notebooks, each responsible for a dedicated stage of the Medallion Architecture. The notebooks dynamically retrieve processing configurations from the metadata file, allowing the same implementation to process multiple datasets while minimizing code duplication and simplifying maintenance.

5.6 Azure SQL Database

Azure SQL Database serves as the reporting repository for the project. After the completion of Gold layer processing, the curated fact and dimension tables are published to Azure SQL Database.

This database provides a structured relational model that can be efficiently queried by reporting tools and business users. It acts as the consumption layer between the data engineering platform and downstream analytical applications.

5.7 Power BI

Power BI is the visualization and reporting component of the solution. It connects to Azure SQL Database to create interactive dashboards and business reports using the curated datasets generated by the Gold layer.

The dashboards enable business users to analyze key performance indicators such as sales performance, customer trends, and product analysis through an intuitive reporting interface.

5.8 Component Integration

The Azure services collectively implement a complete end-to-end data engineering solution. Azure Data Factory orchestrates data ingestion from multiple source systems and initiates processing in Azure Databricks. Azure Data Lake Storage Gen2 stores the data throughout the Medallion Architecture, while Azure Databricks performs metadata-driven processing to

generate validated and business-ready datasets. The curated Gold layer is published to Azure SQL Database, which serves as the reporting source for Power BI dashboards.

The clear separation of responsibilities across these components improves scalability, simplifies maintenance, and enables the platform to support future business and data growth with minimal changes to the overall architecture.

---

6. Data Sources

6.1 Purpose

The Enterprise Retail Analytics Platform integrates data from multiple operational source systems. These source systems provide transactional, master, and reference data required for retail analytics. The data is received in different formats and at different frequencies and is ingested into Azure Data Lake Storage Gen2 using Azure Data Factory.

The project uses the following four source systems.

6.2 Orders Data

Source Type: CSV File

Frequency: Daily

Description:

The Orders dataset represents the primary transactional data of the retail business. It contains details of customer purchases, including product information, quantities, unit prices, and order dates. As the highest volume dataset in the platform, it forms the foundation of sales analytics and is ultimately transformed into the Sales Fact table within the Gold layer.

The dataset is received as a daily CSV file and is processed using an incremental loading strategy based on the configured watermark column. This approach ensures that only newly generated transaction records are processed during each pipeline execution, improving overall processing efficiency while minimizing redundant data movement.

The Orders dataset also establishes relationships with the Customer and Product datasets, enabling the platform to generate integrated business reports across multiple dimensions.

Key Attributes

- OrderID
- CustomerID
- ProductID
- OrderDate
- Quantity
- UnitPrice
- StoreCode

6.3 Products Data

Source Type: CSV File

Frequency: Weekly

Description:

The Products dataset contains the master information associated with every product offered by the retail organization. It provides descriptive attributes such as product name, category, subcategory, brand, and cost price, which are required to enrich sales transactions during analytical processing.

Unlike transactional data, product information changes less frequently and is therefore processed using a full load strategy. The dataset serves as the Product Dimension within the Gold layer and provides the descriptive context required for category-wise, brand-wise, and product-level reporting.

Key Attributes

- ProductID
- ProductName
- Category
- SubCategory
- Brand
- CostPrice

6.4 Customers Data

Source Type: Azure SQL Database

Frequency: Incremental

Description:

The Customers dataset stores the master information associated with retail customers. It includes customer identification details, contact information, and geographical attributes that are required for customer-centric analysis.

Since customer information can change over time, the dataset is processed using a watermark-based incremental loading strategy. The Gold layer further implements Slowly Changing Dimension (SCD) Type 2 processing to preserve historical customer information whenever tracked attributes are modified. This enables historical reporting while maintaining the latest customer information for operational analysis.

The Customers dataset serves as the Customer Dimension within the analytical data model.

Key Attributes

- CustomerID
- FirstName
- LastName
- Email
- Phone
- City
- State
- LastUpdated

6.5 Exchange Rates Data

Source Type: JSON

Frequency: Daily

Description:

The Exchange Rates dataset provides reference data used for currency conversion and analytical reporting. The dataset is available in JSON format and contains exchange rate information for different currencies along with the corresponding effective date.

Unlike the transactional and master datasets, the Exchange Rates dataset represents reference data that can be utilized for reporting scenarios involving multiple currencies. The JSON file is ingested into the platform through the metadata-driven framework and processed using an incremental loading strategy based on the configured watermark column. This approach ensures that only newly available exchange rate records are processed during subsequent pipeline executions.

After validation in the Silver layer, the dataset is stored as the Exchange Rates Dimension in the Gold layer, where it can be used for reporting and future analytical enhancements requiring currency conversion.

Key Attributes

- exchange_info
- rate_details
- metadata
- regions
- quality_checks
- audit_trail

---

7. Solution – Metadata-Driven Framework

7.1 Overview

The Enterprise Retail Analytics Platform is implemented using a metadata-driven framework that enables dynamic, reusable, and scalable data processing across multiple datasets. Instead of embedding dataset-specific logic within Azure Data Factory (ADF) pipelines or Azure Databricks notebooks, all processing configurations are maintained in a centralized JSON metadata file.

The metadata file acts as the control layer of the framework by defining how each dataset should be ingested, validated, transformed, stored, and published. During execution, Azure Data Factory

passes the required runtime parameters, including the dataset name, metadata configuration path, and pipeline run identifier, to the Databricks notebooks. The notebooks dynamically retrieve the corresponding configuration and execute the required processing logic without requiring dataset-specific code.

This configuration-driven approach allows the same Azure Data Factory pipelines and Databricks notebooks to process multiple datasets with different source types, schemas, validation rules, loading strategies, and target tables. As a result, the framework minimizes code duplication, simplifies maintenance, improves scalability, and enables new datasets to be integrated by updating the metadata configuration rather than modifying application code.

The metadata-driven architecture forms the foundation of the entire data engineering solution and controls every stage of the processing lifecycle, including data ingestion, validation, transformation, historical data management, publishing, and audit logging.

7.2 Metadata Architecture

The metadata architecture defines the logical organization of the centralized JSON configuration file used throughout the Enterprise Retail Analytics Platform. The metadata file serves as the central control repository for the framework by maintaining all processing configurations outside the Azure Data Factory pipelines and Azure Databricks notebooks.

Rather than embedding dataset-specific information within the implementation, the framework retrieves all processing instructions from the metadata during runtime. This allows the same notebooks and pipelines to process multiple datasets dynamically based on their individual configurations.

The metadata configuration is organized into five major sections, each responsible for a different aspect of the framework.

Project Configuration

Stores general information related to the project, including the project name, and acts as the root configuration for the entire framework.

Storage Configuration

Defines the Azure Storage Account and Azure Data Lake Storage Gen2 container used for storing Raw, Bronze, Silver, Gold, Audit, and configuration data.

Framework Configuration

Contains common framework settings that are shared across all datasets, including storage paths, audit configuration, audit columns, and common Silver layer processing behaviour.

Publish Configuration

Defines the publishing destination for the curated Gold layer tables, including the target database, schema, destination table names, and write mode used for Azure SQL publishing.

Dataset Configuration

Contains dataset-specific processing rules for every dataset integrated into the framework. Each dataset maintains its own source information, loading strategy, schema definition, validation rules, business rules, Gold layer configuration, and Slowly Changing Dimension (SCD) settings.

By separating framework-level configuration from dataset-level configuration, the architecture enables centralized management of the entire data engineering solution while allowing each dataset to maintain its own processing behaviour. This approach significantly improves scalability, simplifies maintenance, and enables new datasets to be integrated through configuration changes rather than application code modifications.

7.3 Metadata Configuration

The Metadata Configuration is the core component of the metadata-driven framework. It stores all framework-level and dataset-level processing configurations required by the Azure Data Factory pipelines and Azure Databricks notebooks. Rather than embedding source details, validation rules, schemas, target tables, and business logic within the implementation, these configurations are maintained in a centralized JSON metadata file.

During execution, the Databricks notebooks retrieve the required configuration using the dataset name provided by Azure Data Factory. Based on the retrieved metadata, the framework dynamically determines the source location, loading strategy, validation rules, transformation logic, target tables, and publishing configuration for the selected dataset.

The Metadata Configuration is divided into five logical sections:

- Project Configuration
- Storage Configuration
- Framework Configuration
- Publish Configuration
- Dataset Configuration

Each section is responsible for controlling a specific part of the data engineering framework.

7.3.1 Project Configuration

The Project Configuration section contains general information related to the overall data engineering solution. It acts as the root configuration of the metadata file and provides a common identity for the entire framework.

In the current implementation, the project name is defined as **retail_analytics**. This configuration allows the framework to identify the project and maintain consistency across Azure Data Factory pipelines, Azure Databricks notebooks, and supporting framework utilities.

Project Configuration Components

Configuration	Description
Project Name	Unique identifier of the project used throughout the framework

The Project Configuration is typically the smallest section of the metadata file, but it provides the common context under which all datasets and framework components operate.

7.3.2 Storage Configuration

The Storage Configuration defines the Azure Storage Account and Azure Data Lake Storage Gen2 container used by the framework. Instead of hardcoding storage account details within notebooks or pipelines, these values are retrieved dynamically from the metadata configuration.

This centralized approach simplifies maintenance and allows the framework to be migrated to another storage account by updating the metadata without modifying notebook code.

Storage Configuration Components

Configuration	Description
Storage Account	Azure Storage Account used by the project
Container	ADLS Gen2 container storing all project data

The Storage Configuration is used by the framework to construct storage paths for reading and writing datasets across the Raw, Bronze, Silver, Gold, Audit, and Configuration layers.

7.3.3 Framework Configuration

The Framework Configuration defines the common processing behaviour that applies to every dataset processed by the platform. Unlike the Dataset Configuration, which controls individual datasets, the Framework Configuration contains settings that are shared across the entire data engineering solution.

These common settings ensure that every dataset follows a consistent processing standard without requiring duplicate configuration.

The Framework Configuration consists of four major components:

- Target Paths
- Audit Configuration
- Audit Columns
- Silver Layer Configuration

Target Paths

The Target Paths define the standard storage locations used by the framework for each processing layer. Every notebook retrieves these paths dynamically from the metadata while reading or writing datasets.

Layer	Purpose
Bronze	Stores raw Delta tables
Silver	Stores validated Delta tables
Gold	Stores curated analytical tables
Audit	Stores audit logs and rejected records

By centralizing these paths, the framework maintains a consistent storage structure and avoids hardcoded folder locations within the notebooks.

Audit Configuration

The Audit Configuration specifies the storage locations used for audit logs and rejected records generated during pipeline execution.

Configuration	Description
Audit Log Path	Stores execution logs generated during processing
Reject Records Path	Stores records that fail validation during the Silver layer

These audit locations support monitoring, troubleshooting, and operational reporting by maintaining complete execution history for every pipeline run.

Audit Columns

The framework automatically appends technical audit columns to the processed datasets. These columns provide complete traceability by recording information about pipeline execution and processing status.

Audit Column	Description
_AdfPipelineRunId	Unique identifier of the Azure Data Factory pipeline execution
_IngestionTimestamp	Timestamp indicating when the record entered the Bronze layer
_ProcessedTimestamp	Timestamp indicating when the record completed Silver processing
_IsRejected	Indicates whether the record failed validation and was redirected to the rejected records repository

These audit attributes enable data lineage, operational monitoring, and troubleshooting throughout the processing lifecycle.

Silver Layer Configuration

The Silver Layer Configuration stores common validation behaviour that applies to every dataset processed within the Silver layer.

The framework supports centralized configuration for:

- Duplicate removal
- Primary key validation
- Mandatory column validation
- Invalid record removal
- String trimming
- Date standardization
- Business rule validation
- Rejected record handling
- Default null handling strategy

Instead of implementing these settings individually for every dataset, the framework retrieves them from the metadata and applies them consistently during Silver layer processing. This centralized approach improves maintainability and ensures that common validation behaviour remains consistent across all datasets.

7.3.4 Publish Configuration

The Publish Configuration controls how the curated Gold layer datasets are published to Azure SQL Database after the completion of Gold layer processing. Instead of hardcoding the publishing logic within Azure Data Factory pipelines or Azure Databricks notebooks, all publishing-related information is maintained within the metadata configuration.

During execution, the Gold notebook reads the Publish Configuration to determine which datasets should be published, the destination database schema, target table names, and write mode. Only datasets marked as enabled are published to Azure SQL Database.

Centralizing the publishing configuration provides flexibility by allowing reporting tables to be modified, enabled, or disabled without changing the notebook implementation.

Publish Configuration Components

Configuration	Description
Target	Specifies the publishing destination (Azure SQL Database)
Default Schema	Database schema used for all published tables
Dataset Name	Identifies the Gold dataset to be published
Table Name	Target table name in Azure SQL Database
Enabled	Determines whether the dataset should be published
Write Mode	Specifies the write operation such as Overwrite or Append

For every dataset, the framework dynamically reads the corresponding publishing configuration and transfers the processed Gold layer data into Azure SQL Database. Since all publishing behaviour is maintained within the metadata, new reporting tables can be introduced simply by updating the configuration file without modifying the notebook code.

7.3.5 Dataset Configuration

The Dataset Configuration is the most important section of the metadata file because it defines the complete processing behaviour for every dataset integrated into the platform. Each dataset maintains an independent configuration block that specifies how it should be ingested, validated, transformed, and published.

When Azure Data Factory executes a notebook, it passes the dataset name as a runtime parameter. The notebook retrieves the corresponding dataset configuration using the `get_dataset_config()` framework function and dynamically applies the required processing logic. This allows a single reusable notebook implementation to process multiple datasets with different source types and processing requirements.

Each dataset configuration consists of the following components.

Dataset Information

The Dataset Information section defines the basic properties required to identify and process a dataset.

Configuration	Description
Dataset Name	Unique identifier of the dataset
Active	Enables or disables dataset processing
Execution Order	Defines the sequence in which datasets are processed
Source Type	Identifies whether the source is SQL, CSV, or REST API
Source Path	Specifies the location of the source data
File Format	Defines the input file format such as CSV or JSON

These properties enable the framework to identify the correct source dataset and execute the appropriate ingestion logic.

Primary Key and Relationships

The metadata also defines the business keys required for validation and analytical processing.

Primary Key

The Primary Key uniquely identifies each record within a dataset and is primarily used for duplicate detection during Silver layer processing.

Examples include:

- CustomerID
- ProductID
- OrderID

For datasets such as Exchange Rates, multiple columns together form a composite primary key.

Relationships

Relationships define the foreign key associations between datasets.

For example:

- Orders → Customers
- Orders → Products

These relationships enable the framework to validate referential integrity during Silver layer processing and establish fact-to-dimension relationships during Gold layer processing.

Processing Configuration

The Processing Configuration controls how each dataset is ingested and processed.

Configuration	Description
Load Type	Specifies Full Load or Incremental Load
Watermark Column	Used for watermark-based incremental loading
Partition Columns	Defines columns used for data partitioning
Flatten Required	Indicates whether nested JSON structures should be flattened

Datasets such as Customers and Orders use watermark-based incremental loading, while Products and Exchange Rates are processed using full load. For JSON datasets, the framework automatically executes the flattening process whenever the **Flatten Required** property is enabled.

Validation Rules

The Validation Rules section defines the common data quality validations that must be performed during Silver layer processing.

The framework supports the following validations:

Validation	Purpose
Mandatory Columns	Ensures required fields are populated
Numeric Columns	Validates numeric values
Date Columns	Validates date values
Timestamp Columns	Validates timestamp values
Email Columns	Validates email format

Validation	Purpose
Positive Value Columns	Ensures configured numeric values are greater than zero

These validations are applied dynamically based on the metadata configuration without requiring dataset-specific validation code.

Business Rules

Business Rules define dataset-specific validations that extend the common data quality checks.

Examples implemented in this project include:

- Customer names must contain only alphabetic characters.
- Email addresses must follow a valid email format.
- Phone numbers must contain exactly ten digits.
- Currency codes must follow the ISO three-letter format.
- Quantity and Unit Price must be greater than zero.
- Store codes must follow the configured naming pattern.

These rules are retrieved dynamically from the metadata and executed during Silver layer processing.

Schema Definition

Each dataset contains its own schema definition within the metadata.

The schema specifies:

Configuration	Description
Column Name	Name of the column
Data Type	Expected data type
Nullable	Indicates whether null values are permitted

During Silver layer processing, the framework reads the schema configuration and converts every column to its expected data type dynamically. This eliminates hardcoded schema definitions from the notebook implementation and allows different datasets to maintain independent schema structures.

Gold Configuration

The Gold Configuration determines how validated datasets are transformed into analytical tables.

Configuration	Description
Table Type	Specifies Fact or Dimension processing
Fact Name	Analytical fact table name
Surrogate Key	Generated surrogate key for dimensions

Based on this configuration, the Gold notebook determines whether the dataset should be processed as a Fact table or a Dimension table.

SCD Configuration

The SCD Configuration controls historical data management for dimension datasets.

The configuration specifies:

Configuration	Description
Enabled	Enables or disables SCD processing
SCD Type	Specifies the SCD implementation type
Tracked Columns	Identifies columns monitored for historical changes

In the current implementation, the Customers dataset is configured with Slowly Changing Dimension (SCD) Type 2. Whenever changes occur in the tracked attributes (such as **City** or **State**), the framework preserves the historical record by creating a new version while retaining the previous version for historical analysis.

7.4 How Metadata Drives Notebook Execution

The Enterprise Retail Analytics Platform uses a metadata-driven execution model in which Azure Data Factory orchestrates the pipeline execution while Azure Databricks performs all data processing dynamically based on the metadata configuration.

Unlike traditional ETL implementations where separate notebooks are developed for each dataset, the proposed framework uses a single reusable notebook for each processing layer. The

notebook determines its execution behaviour by reading the corresponding dataset configuration from the centralized metadata file.

During pipeline execution, Azure Data Factory invokes the Databricks notebook and passes the following runtime parameters:

Runtime Parameter	Description
Dataset Name	Identifies the dataset to be processed
Metadata Configuration Path	Location of the centralized metadata JSON file
Pipeline Run Identifier	Unique Azure Data Factory pipeline execution identifier

After receiving these parameters, the notebook invokes the **get_dataset_config()** framework function. This function searches the metadata configuration and retrieves the configuration corresponding to the selected dataset.

The retrieved metadata contains all information required for processing, including the source type, source location, loading strategy, schema definition, validation rules, business rules, target tables, Gold layer configuration, publishing configuration, and Slowly Changing Dimension settings.

Based on this information, the notebook dynamically determines the processing logic without requiring any dataset-specific implementation. The same notebook can therefore process Customers, Products, Orders, Exchange Rates, or any newly onboarded dataset by simply reading a different metadata configuration.

The execution process can be summarized as follows:

1. Azure Data Factory starts the pipeline execution.
2. Runtime parameters are passed to the Databricks notebook.
3. The notebook retrieves the dataset configuration using the **get_dataset_config()** function.
4. The framework identifies the source information, schema, validation rules, loading strategy, and target tables.
5. The notebook executes the required ingestion, validation, transformation, or publishing logic based on the processing layer.
6. The processed data is written to the configured target location, and audit information is updated.

This execution model separates configuration from implementation, allowing the framework to process multiple datasets using the same reusable notebooks. As a result, introducing a new dataset requires only a metadata update, while the existing Azure Data Factory pipelines and Databricks notebooks continue to operate without modification.

7.4.1 Reusable Framework Functions

To eliminate code duplication and improve maintainability, the Enterprise Retail Analytics Platform is implemented using a reusable framework consisting of common utility functions. Instead of implementing dataset-specific logic within every notebook, these reusable functions perform common processing tasks such as reading metadata, loading data, applying audit information, handling incremental loading, writing Delta tables, and generating audit logs.

Each processing layer invokes the required framework functions depending on its responsibility. Since these functions are generic and metadata-driven, the same implementation can process multiple datasets without requiring code modifications.

The following table summarizes the reusable framework functions implemented in the project.

Function	Processing Layer	Purpose
`get_dataset_config()`	Bronze, Silver, Gold	Reads dataset-specific configuration from the metadata file.
`get_active_datasets()`	ADF / Framework	Retrieves all active datasets configured for processing.
`read_source_file()`	Bronze	Reads source data from Azure SQL Database, CSV files, or JSON files based on the metadata configuration.
`read_delta()`	Silver, Gold	Reads Delta tables from the previous processing layer.
`get_watermark()`	Bronze	Retrieves the previously processed watermark value for incremental datasets.
`update_watermark()`	Bronze	Updates the latest processed watermark after successful execution.
`flatten_df()`	Silver	Flattens nested JSON structures whenever the metadata indicates flattening is required.
`add_audit_columns()`	Bronze, Silver	Appends audit columns such as Pipeline Run ID, Ingestion Timestamp, Processed Timestamp, and Rejected Flag.
`write_delta()`	Bronze, Silver, Gold	Writes processed datasets into Delta format within Azure Data Lake Storage Gen2.

Function	Processing Layer	Purpose
`write_audit_log()`	Bronze, Silver, Gold	Records execution details including record counts, processing status, and timestamps for monitoring purposes.

These reusable framework functions form the foundation of the metadata-driven architecture and are shared across all processing layers. Instead of implementing separate logic for each dataset, the framework dynamically retrieves the required metadata and invokes the appropriate utility functions, ensuring consistent processing behaviour throughout the platform.

7.4.2 Framework Function Responsibilities

Each framework function performs a specific task within the overall data processing lifecycle.

get_dataset_config()

This function retrieves the dataset-specific configuration from the metadata file using the dataset name provided by Azure Data Factory. The returned configuration includes source information, validation rules, schema definitions, loading strategy, target tables, and Gold layer settings. Every processing notebook invokes this function at the beginning of execution to determine how the selected dataset should be processed.

get_active_datasets()

This function identifies all datasets that are marked as active within the metadata configuration. Azure Data Factory uses this information to determine which datasets should be included during pipeline execution.

read_source_file()

This function dynamically reads source data based on the configured source type. Depending on the metadata, it connects to Azure SQL Database, reads CSV files from Azure Data Lake Storage Gen2, or retrieves JSON data from REST API sources. This generic implementation allows the Bronze notebook to process different source systems using the same function.

read_delta()

This function reads Delta tables generated by the previous processing layer. The Silver notebook retrieves Bronze Delta tables, while the Gold notebook retrieves validated Silver Delta tables for further transformation.

get_watermark()

For datasets configured with incremental loading, this function retrieves the previously processed watermark value. The retrieved watermark is used to identify newly inserted or modified records, ensuring that only incremental changes are processed during subsequent executions.

update_watermark()

After successful processing of an incremental dataset, this function identifies the latest watermark value and stores it for future executions. This enables efficient incremental processing while preventing duplicate data ingestion.

flatten_df()

Certain datasets, such as JSON files received from REST APIs, contain nested structures that cannot be directly processed. Whenever the metadata specifies `flatten_required = true`, this function converts nested JSON attributes into a tabular format suitable for validation and analytical processing.

add_audit_columns()

This function appends technical audit columns to every processed record. These columns capture information such as the Azure Data Factory Pipeline Run ID, ingestion timestamp, processed timestamp, and rejected record indicator. Audit columns provide complete traceability and support monitoring throughout the processing lifecycle.

write_delta()

After successful processing, this function writes the resulting DataFrame to Azure Data Lake Storage Gen2 in Delta format. The destination path is determined dynamically from the metadata configuration, allowing the same function to write Bronze, Silver, and Gold datasets.

write_audit_log()

This function records execution details after each processing stage. The audit log includes information such as source record count, processed record count, rejected record count, execution status, processing timestamps, and pipeline execution identifiers. These logs support operational monitoring, troubleshooting, and execution auditing.

7.5 Adding a New Dataset

The metadata-driven framework allows new datasets to be integrated without modifying Azure Data Factory pipelines or Azure Databricks notebooks.

To onboard a new dataset, the following steps are performed:

Step 1: Upload the source dataset into the Raw layer of Azure Data Lake Storage Gen2 using ADF.

Step 2: Add a new dataset entry in the metadata configuration by specifying the dataset name, source type, source path, file format, execution order, primary key, load strategy, watermark column (if incremental), and target table names.

Step 3: Define the schema by specifying each column name, expected data type, and nullable constraint.

Step 4: Configure dataset-specific validation rules, including mandatory columns, numeric columns, date or timestamp columns, email columns, positive value validations, and any required business rules.

Step 5: Configure Gold layer settings by specifying whether the dataset should be processed as a Fact or Dimension table. If historical tracking is required, enable SCD processing and define the tracked columns.

Step 6: Execute the existing Azure Data Factory pipeline. The framework automatically detects the new metadata entry and processes the dataset using the reusable Databricks notebooks without requiring any code changes.

This configuration-driven approach significantly reduces development effort, improves maintainability, and enables rapid onboarding of new datasets.

---

8. Data Ingestion

8.1 Overview

Data ingestion is the first operational phase of the Enterprise Retail Analytics Platform and is responsible for collecting data from multiple heterogeneous source systems and preparing it for downstream analytical processing. The platform integrates structured and semi-structured data from Azure SQL Database, CSV files, and REST API sources using Azure Data Factory (ADF) as the orchestration service.

Azure Data Factory extracts data from the configured source systems and stores it in the Raw layer of Azure Data Lake Storage Gen2 (ADLS Gen2). Once the source data has been successfully ingested, Azure Data Factory invokes the Azure Databricks notebooks, which process the data through the Bronze, Silver, and Gold layers of the Medallion Architecture.

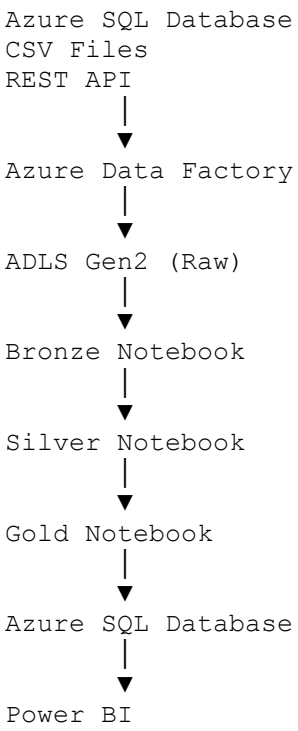
Unlike traditional ETL implementations where separate notebooks are developed for every dataset, this project uses the metadata-driven framework described in Chapter 7. During execution, each notebook dynamically retrieves the required dataset configuration from the

centralized metadata file and determines the appropriate processing logic. As a result, the same notebooks can process multiple datasets without requiring dataset-specific implementations.

The data ingestion framework supports both Full Load and Incremental Load strategies. Static reference datasets such as Products are processed using Full Load, while frequently changing datasets such as Customers and Orders use watermark-based Incremental Loading. Exchange Rates are ingested from a REST API and processed according to the configured metadata.

Throughout the execution process, the framework captures audit information, maintains execution logs, records rejected data, and updates watermark values to support operational monitoring, troubleshooting, and incremental processing. After successful processing, the curated datasets are published to Azure SQL Database for business reporting and visualization through Power BI.

8.2 Data Ingestion Architecture



The data ingestion architecture illustrates the complete execution flow of the Enterprise Retail Analytics Platform. Azure Data Factory acts as the orchestration layer responsible for extracting data from multiple operational systems and coordinating the execution of the processing notebooks.

Initially, data from Azure SQL Database, CSV files, and REST API sources is copied into the Raw layer of Azure Data Lake Storage Gen2. The Raw layer preserves the original source data and acts as the landing zone for downstream processing.

After the ingestion phase, Azure Data Factory invokes the Azure Databricks notebooks sequentially. The Bronze notebook converts the raw data into Delta format while preserving the original records. The Silver notebook applies schema standardization, data validation, duplicate removal, and business rule validation to generate high-quality datasets. Finally, the Gold notebook transforms the validated datasets into business-ready fact and dimension tables, applies Slowly Changing Dimension (SCD) processing where required, and prepares the data for reporting.

The curated Gold layer datasets are then published to Azure SQL Database, where they become available for Power BI dashboards and business reporting.

8.3 End-to-End Processing Flow

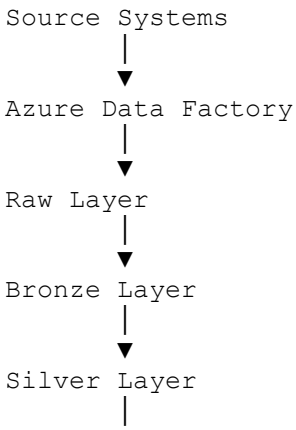
The Enterprise Retail Analytics Platform follows a sequential processing model in which every dataset progresses through a series of well-defined stages before becoming available for business reporting.

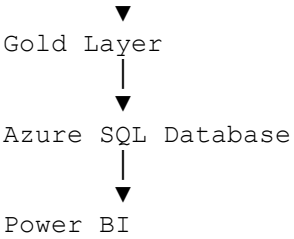
The processing begins when Azure Data Factory extracts data from the configured source systems and stores it within the Raw layer of Azure Data Lake Storage Gen2. Once the source data has been successfully copied, Azure Data Factory invokes the reusable Databricks notebooks by passing the dataset name, metadata configuration path, and pipeline execution identifier.

Each notebook retrieves the corresponding dataset configuration from the metadata file using the framework utilities described in Chapter 7. Based on the retrieved configuration, the framework determines the source location, loading strategy, validation rules, schema definition, target tables, and publishing behaviour required for the selected dataset.

The data is then processed sequentially through the Bronze, Silver, and Gold layers. During each stage, audit information is recorded and processing statistics are captured to ensure complete traceability. After the Gold layer processing is completed, the curated datasets are published into Azure SQL Database and become available for analytical reporting through Power BI.

The complete execution sequence is illustrated below.





8.4 Bronze Layer

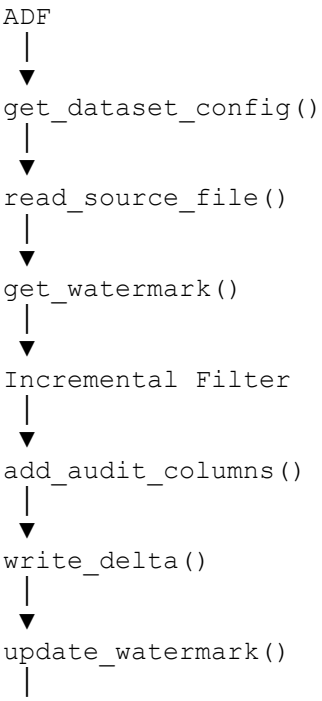
8.4.1 Purpose

The Bronze layer is the first processing layer of the Medallion Architecture and serves as the raw data repository for the platform. Its primary objective is to ingest source data into Delta format while preserving the original records without applying business transformations or data quality validations.

Since the Bronze layer maintains an exact copy of the source data, it provides complete data lineage and enables data recovery whenever downstream processing requires re-execution.

The Bronze notebook is implemented as a generic metadata-driven notebook capable of processing multiple datasets. Rather than containing dataset-specific logic, it retrieves all required processing information dynamically from the metadata configuration.

8.4.2 Bronze Layer Execution Flow



▼

`write_audit_log()`

Figure 8.2 Bronze Layer Execution Flow

8.4.3 Bronze Processing Steps

The Bronze notebook performs the following processing sequence:

Step 1 – Read Dataset Configuration

The notebook begins by invoking the `get_dataset_config()` framework function to retrieve the metadata associated with the selected dataset. The metadata provides information such as source type, source path, loading strategy, watermark column, and Bronze target table.

Step 2 – Read Source Data

The `read_source_file()` framework function reads data from the configured source system. Depending on the metadata, the function connects to Azure SQL Database, reads CSV files from Azure Data Lake Storage Gen2, or loads JSON data retrieved from the REST API.

Step 3 – Apply Incremental Loading

For datasets configured with Incremental Load, the notebook retrieves the previously processed watermark using the `get_watermark()` function. Records with watermark values greater than the stored watermark are selected for processing, while previously processed records are ignored. Full Load datasets bypass this step and process the complete dataset.

Step 4 – Append Audit Columns

The `add_audit_columns()` function appends technical metadata to every processed record. These audit attributes include the Azure Data Factory Pipeline Run Identifier and the ingestion timestamp, enabling complete data lineage.

Step 5 – Store Bronze Data

The processed DataFrame is written to Azure Data Lake Storage Gen2 in Delta format using the `write_delta()` framework function. The destination path is determined dynamically from the metadata configuration.

Step 6 – Update Watermark

After successful processing, the `update_watermark()` function records the latest watermark value. This value is used during the next execution to identify newly inserted or modified records.

Step 7 – Generate Audit Log

Finally, the notebook invokes the `write_audit_log()` function to capture execution statistics such as source record count, processed record count, execution status, processing timestamps, and pipeline execution identifier.

8.4.4 Incremental Loading

The Bronze framework supports both Full Load and Incremental Load processing based on the dataset configuration stored in the metadata file.

For datasets configured with Incremental Load, the framework retrieves the previously processed watermark value and compares it with the incoming data. Only records with watermark values greater than the stored watermark are processed. This minimizes unnecessary data movement, improves pipeline performance, and prevents duplicate ingestion.

Datasets configured for Full Load process the complete source dataset during every execution.

8.4.5 Audit Framework

The Bronze layer automatically records audit information for every pipeline execution. Audit columns appended to the processed records provide complete traceability throughout the processing lifecycle.

In addition to record-level audit information, the framework maintains execution logs containing processing status, source record count, target record count, execution timestamps, and watermark information. These logs simplify operational monitoring and troubleshooting.

8.4.6 Bronze Output

After successful execution, the Bronze notebook generates Delta tables within the Bronze layer of Azure Data Lake Storage Gen2. These tables preserve the original source records while enriching them with audit information.

The Bronze layer acts as the foundation for downstream processing, providing reliable input for the Silver layer where data quality validation and business transformations are performed.

8.5 Silver Layer

8.5.1 Purpose

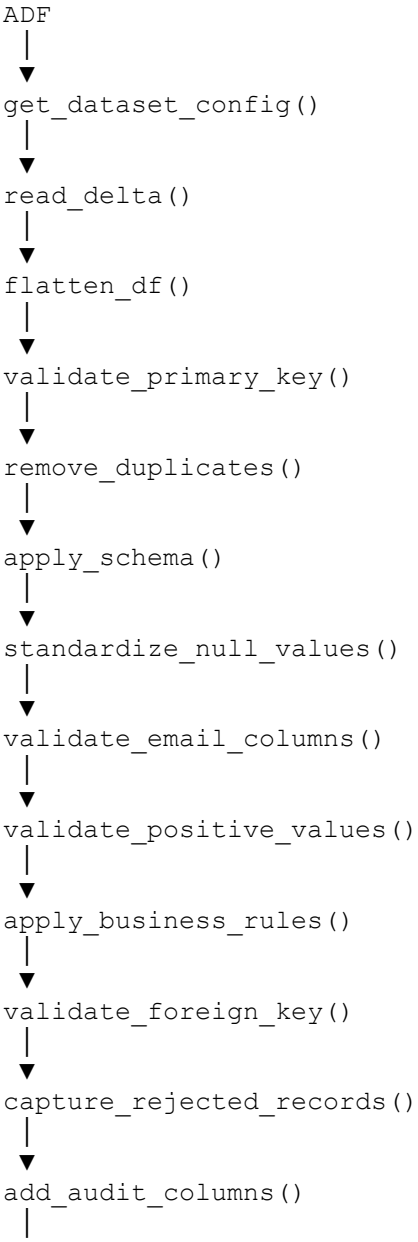
The Silver layer is the second stage of the Medallion Architecture and is responsible for transforming raw Bronze data into validated, standardized, and high-quality datasets suitable for analytical processing. Unlike the Bronze layer, which preserves the original source data, the

Silver layer applies data cleansing, schema standardization, validation rules, duplicate removal, business rule validation, and rejected record handling.

The Silver notebook is implemented as a generic metadata-driven notebook capable of processing multiple datasets. During execution, the notebook dynamically retrieves the dataset configuration from the metadata file and applies the required processing logic based on the configured schema, validation rules, business rules, and processing settings.

The output of the Silver layer consists of validated Delta tables that serve as the input for the Gold layer.

8.5.2 Silver Layer Execution Flow



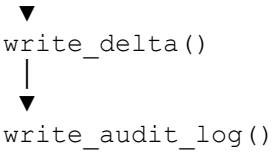


Figure 8.3: Silver Layer Execution Flow

8.5.3 Silver Processing Steps

The Silver notebook executes a sequence of metadata-driven processing steps to improve the quality and consistency of the Bronze data before it is passed to the Gold layer.

Step 1 – Read Dataset Configuration

The notebook begins by retrieving the dataset configuration from the metadata file using the dataset name provided by Azure Data Factory. The configuration contains the schema definition, validation rules, business rules, partition information, target tables, and processing settings required for the selected dataset.

Step 2 – Read Bronze Delta Table

The framework reads the corresponding Bronze Delta table from Azure Data Lake Storage Gen2 using the metadata-defined target path. This dataset becomes the input for all subsequent validation and transformation activities.

Step 3 – Flatten Nested JSON Structures

Datasets received in nested JSON format, such as the Exchange Rates dataset, require structural transformation before validation.

Whenever the metadata specifies `flatten_required = true`, the framework automatically flattens nested objects and arrays into a relational tabular structure suitable for further processing. Datasets that do not require flattening bypass this step.

Step 4 – Validate Primary Key

The framework validates the configured primary key to ensure that every record contains a valid business identifier.

Records with missing or invalid primary key values are identified during this stage and marked for rejection.

Step 5 – Remove Duplicate Records

Duplicate records are removed using the primary key defined in the metadata configuration.

This process guarantees that only one valid version of each business record progresses to the Gold layer, preventing duplicate analytical results.

Step 6 – Apply Schema Standardization

The framework dynamically reads the schema definition from the metadata configuration and converts each column into its expected data type.

Instead of hardcoding schemas within the notebook, every dataset maintains its own schema configuration inside the metadata. This enables the same notebook implementation to process datasets with completely different structures.

Schema standardization also ensures consistent data types before validation and transformation.

Step 7 – Standardize Invalid Values

The framework identifies invalid string values and converts them into standardized NULL values according to the configured null handling strategy.

This process improves data consistency and simplifies downstream validation and analytical processing.

Step 8 – Validate Email Columns

Columns configured as email fields are validated using regular expression patterns defined by the framework.

Records containing invalid email formats are standardized according to the configured validation strategy, ensuring that only valid email addresses are retained for analytical processing.

Step 9 – Validate Positive Numeric Values

Business measures such as Quantity, Unit Price, Cost Price, and Exchange Rate must contain positive values.

The framework validates these fields dynamically based on the Positive Value Configuration defined in the metadata.

Step 10 – Apply Business Rules

The framework executes dataset-specific business rules maintained within the metadata configuration.

Examples implemented in this project include:

- Customer names must contain only alphabetic characters.
- Phone numbers must contain exactly ten digits.
- Email addresses must follow the configured email format.
- Currency codes must follow the ISO three-letter standard.
- Store codes must satisfy the configured naming pattern.
- Quantity and Unit Price must be greater than zero.

Since these rules are stored within the metadata configuration, new business rules can be introduced without modifying notebook code.

Step 11 – Validate Foreign Key Relationships

Datasets containing relationships are validated using the relationship definitions maintained in the metadata configuration.

For example, the Orders dataset verifies that every CustomerID exists within the Customer dataset and every ProductID exists within the Product dataset before progressing to the Gold layer.

This validation maintains referential integrity across the analytical model.

Step 12 – Capture Rejected Records

Records that fail any validation or business rule are separated from the valid dataset.

Instead of discarding invalid records, the framework stores them within the configured rejected records location together with the corresponding rejection reason.

This approach supports data quality monitoring while preserving failed records for future analysis and correction.

Step 13 – Append Audit Information

After successful validation, the framework appends audit columns such as Processed Timestamp and Rejected Record Indicator to maintain complete processing lineage.

Step 14 – Write Silver Delta Tables

Validated records are written into Azure Data Lake Storage Gen2 as Delta tables.

The destination location is determined dynamically from the metadata configuration, allowing the same notebook to process multiple datasets without modification.

Step 15 – Generate Audit Log

Finally, the framework records execution statistics including source record count, validated record count, rejected record count, processing status, execution timestamps, and pipeline execution identifier.

These logs provide complete operational visibility and support troubleshooting.

8.5.4 Data Quality Validation

The Silver layer implements centralized data quality management using the validation rules maintained within the metadata configuration.

The implemented validation framework includes:

- Primary Key Validation
- Duplicate Removal
- Schema Standardization
- Mandatory Column Validation
- Numeric Validation
- Date Validation
- Timestamp Validation
- Email Validation
- Positive Value Validation
- Foreign Key Validation
- Business Rule Validation

Since these validations are metadata-driven, changes to validation logic require only metadata updates rather than notebook modifications.

8.5.5 Rejected Records

Records failing validation or business rule checks are redirected to the rejected records repository instead of being discarded.

Each rejected record contains both the original data and the corresponding rejection reason, enabling data stewards to analyse, correct, and reload invalid records without affecting valid analytical datasets.

8.5.6 Silver Layer Output

After successful execution, the Silver notebook produces validated and standardized Delta tables within the Silver layer of Azure Data Lake Storage Gen2.

These datasets contain consistent schemas, validated business data, enforced referential integrity, and complete audit information. The Silver layer serves as the trusted source for Gold layer

transformations, ensuring that only high-quality data is used to generate business-ready fact and dimension tables.

8.6 Gold Layer

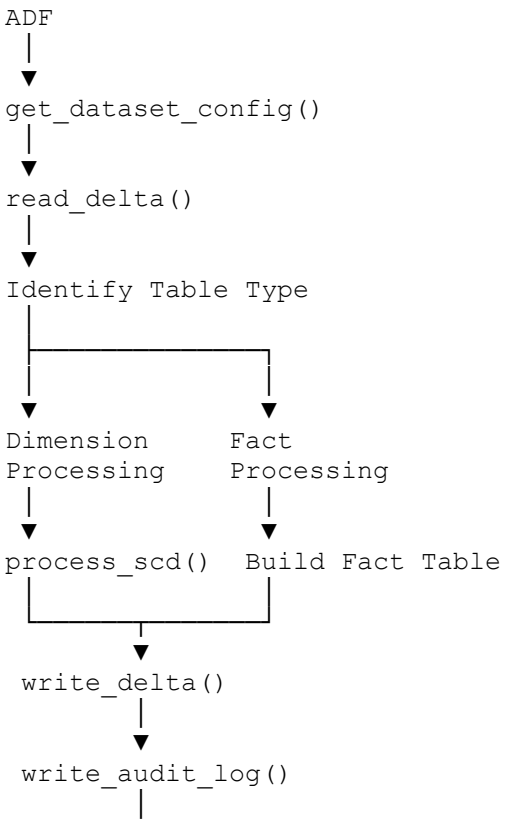
8.6.1 Purpose

The Gold layer is the final stage of the Medallion Architecture and is responsible for transforming validated Silver layer datasets into business-ready analytical tables. These tables are optimized for reporting, business intelligence, and downstream analytical applications.

Unlike the Bronze and Silver layers, which focus on data ingestion and quality improvement, the Gold layer organizes the processed data into Fact and Dimension tables that support analytical queries and reporting requirements.

The Gold notebook is implemented as a reusable metadata-driven notebook that dynamically determines how each dataset should be processed. Based on the metadata configuration, the framework identifies whether the dataset represents a Fact table or a Dimension table, applies the appropriate transformation logic, performs Slowly Changing Dimension (SCD) processing when required, and prepares the curated datasets for publishing to Azure SQL Database.

8.6.2 Gold Layer Execution Flow



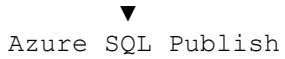


Figure 8.4: Gold Layer Execution Flow

8.6.3 Gold Processing Steps

The Gold notebook performs the following sequence of operations to generate analytical datasets.

Step 1 – Read Dataset Configuration

The notebook retrieves the dataset configuration from the metadata file. The configuration identifies:

- Table Type
- Target Table
- Surrogate Key
- SCD Configuration
- Relationships
- Publishing Configuration

These settings determine how the selected dataset should be processed.

Step 2 – Read Silver Delta Table

The validated dataset generated by the Silver layer is loaded into the Gold notebook.

Since the Silver layer has already completed schema standardization, validation, duplicate removal, and business rule validation, the Gold notebook focuses only on business transformations.

Step 3 – Identify Table Type

The framework examines the **table_type** property stored in the metadata.

Depending on the configuration, the dataset is processed as either:

- Dimension Table
- Fact Table

This allows a single notebook to process different analytical structures dynamically.

8.6.4 Dimension Processing

Datasets configured as **Dimension Tables** are transformed into descriptive business entities used for reporting.

The current implementation processes:

- Customers
- Products
- Exchange Rates

as Dimension tables.

During Dimension processing, the framework:

- Creates surrogate keys (where configured)
- Maintains descriptive attributes
- Preserves business keys
- Stores analytical dimensions within the Gold layer

The target Dimension table is determined dynamically from the metadata configuration.

Customer Dimension

The Customer dataset is configured as a Slowly Changing Dimension (SCD) Type 2.

Whenever tracked attributes such as **City** or **State** are modified, the framework:

- Marks the existing record as inactive.
- Updates the Effective End Date.
- Inserts a new version of the customer record.
- Marks the new record as the current version.

This preserves historical customer information while maintaining the latest version for reporting.

Product Dimension

The Product dataset is processed as a standard Dimension table.

Since historical tracking is not required, the framework stores the latest validated product information without creating historical versions.

Exchange Rate Dimension

The Exchange Rates dataset is processed as a standard Dimension table after the JSON structure has been flattened and validated in the Silver layer.

The resulting dimension provides reference data that can be used for analytical reporting involving multiple currencies.

8.6.5 Fact Table Processing

Datasets configured as **Fact Tables** contain measurable business events.

In the current implementation, the Orders dataset is transformed into the **Sales Fact** table.

The framework dynamically identifies the Orders dataset as a Fact table using the metadata configuration and creates the analytical sales table.

The Sales Fact table contains transactional information such as:

- OrderID
- CustomerID
- ProductID
- OrderDate
- Quantity
- UnitPrice
- StoreCode

The fact table establishes relationships with the Customer, Product, and Exchange Rate dimensions, enabling multidimensional business analysis.

8.6.6 Slowly Changing Dimension (SCD Type 2)

The metadata-driven framework supports Slowly Changing Dimension (SCD) processing for datasets requiring historical tracking.

The Customers dataset is configured with:

- SCD Enabled
- SCD Type 2
- Tracked Columns (City, State)

During execution, the framework compares the incoming Silver record with the existing Gold record.

If changes are detected in the configured tracked columns:

- the existing record is closed,
- a new surrogate key is generated,
- a new version of the record is inserted,
- effective start and end dates are maintained,
- the current record indicator is updated.

This implementation preserves historical customer information while allowing reports to access both current and historical data.

8.6.7 Gold Layer Output

After successful processing, the Gold notebook generates business-ready analytical datasets in Azure Data Lake Storage Gen2.

The Gold layer contains:

Dataset	Table Type
Customers	Dimension
Products	Dimension
Exchange Rates	Dimension
Orders	Fact

These datasets represent the final curated analytical model used for reporting and business intelligence.

The processed Gold tables are subsequently published to Azure SQL Database according to the publishing configuration defined in the metadata file, making them available for visualization through Power BI dashboards.

---

9. Data Publishing and Reporting

9.1 Overview

The Data Publishing and Reporting layer is the final stage of the Enterprise Retail Analytics Platform. After the Gold layer generates curated analytical datasets, the framework publishes the processed Fact and Dimension tables to Azure SQL Database, where they become available for reporting and business intelligence.

The publishing process is implemented using Azure Data Factory and is controlled by the metadata-driven framework. Rather than hardcoding destination tables within the implementation, the framework dynamically retrieves the publishing configuration from the metadata file and transfers only the enabled datasets to Azure SQL Database.

Azure SQL Database serves as the reporting repository for the solution and provides a structured relational model optimized for analytical queries. Power BI connects to Azure SQL Database to create interactive dashboards and business reports using the curated datasets generated by the Gold layer.

This architecture separates analytical processing from reporting, improving performance, maintainability, and scalability while enabling business users to access reliable and business-ready information.

9.2 Gold to Azure SQL Publishing

After the successful completion of Gold layer processing, Azure Data Factory initiates the publishing pipeline responsible for transferring the curated analytical datasets from Azure Data Lake Storage Gen2 into Azure SQL Database.

The publishing pipeline follows a metadata-driven approach. During execution, it retrieves the Publish Configuration from the metadata file and identifies:

- Target Database
- Database Schema
- Target Table Name
- Write Mode
- Enabled Datasets

Only datasets marked as enabled are published.

For every configured dataset, the pipeline performs the following sequence:

Step 1

Read the Publish Configuration.

Step 2

Identify the corresponding Gold Delta table.

Step 3

Read the Gold dataset.

Step 4

Connect to Azure SQL Database.

Step 5

Create or overwrite the destination table based on the configured write mode.

Step 6

Load the curated data into Azure SQL Database.

Step 7

Generate execution logs for monitoring and troubleshooting.

This configuration-driven approach enables publishing behavior to be modified by updating the metadata configuration without requiring changes to the pipeline implementation.

9.3 Azure SQL Database

Azure SQL Database serves as the reporting repository for the Enterprise Retail Analytics Platform.

Unlike Azure Data Lake Storage Gen2, which stores data in Delta format for distributed processing, Azure SQL Database provides a relational structure optimized for reporting and business intelligence applications.

The platform publishes the following analytical tables into Azure SQL Database.

Table	Type
dim_customers	Dimension
dim_products	Dimension
dim_exchange_rates	Dimension
fact_sales	Fact

These tables represent the final curated analytical model consumed by downstream reporting applications.

Since Azure SQL Database provides a familiar relational interface, it enables efficient querying, simplified dashboard development, and integration with reporting tools such as Power BI.

9.4 Power BI Integration

Power BI serves as the visualization layer of the Enterprise Retail Analytics Platform.

Instead of connecting directly to Azure Data Lake Storage Gen2, Power BI retrieves curated analytical data from Azure SQL Database. This architecture isolates reporting from the processing environment and improves dashboard performance.

After connecting to Azure SQL Database, Power BI imports the published Fact and Dimension tables and establishes relationships between them using the business keys generated during Gold layer processing.

The reporting model consists of:

- Sales Fact Table
- Customer Dimension
- Product Dimension
- Exchange Rate Dimension

These relationships enable multidimensional analysis and interactive reporting across various business perspectives.

9.5 Retail Sales Dashboard

The Retail Sales Dashboard provides business users with an interactive visualization of the processed analytical data.

The dashboard presents key business metrics generated from the Gold layer and enables users to analyse sales performance, customer behaviour, and product trends.

The implemented dashboard includes visualizations for:

- Total Sales
- Total Orders
- Revenue by Product
- Revenue by Customer
- Revenue by Store
- Monthly Sales Trend
- Customer Distribution
- Product Performance

Interactive filters allow users to analyse data based on products, customers, stores, and time periods.

Since the dashboard is built on the curated Gold layer, business users access validated and consistent information without interacting directly with the raw processing layers.

9.6 Benefits of the Reporting Layer

The reporting layer provides several advantages for business users and data analysts.

The implemented architecture offers the following benefits:

- Centralized reporting using Azure SQL Database.
- Business-ready analytical datasets generated through the Gold layer.
- Interactive Power BI dashboards for business users.
- Separation between processing and reporting workloads.
- Improved dashboard performance through relational modelling.
- Easy integration with additional reporting tools.
- Metadata-driven publishing without modifying pipeline logic.
- Scalable architecture for future reporting requirements.

---

10. Project Outcome

The Enterprise Retail Analytics Platform was successfully implemented as an end-to-end Azure Data Engineering solution using Azure Data Factory, Azure Databricks, Azure Data Lake Storage Gen2, Azure SQL Database, and Power BI. The solution integrates data from multiple heterogeneous source systems, including CSV files, Azure SQL Database, and REST APIs, into a centralized data platform.

A metadata-driven framework was developed to automate data ingestion and processing. The framework dynamically reads dataset configurations from a centralized metadata file, enabling the same processing logic to be reused across multiple datasets. This approach improves scalability, reduces code duplication, and simplifies the onboarding of new datasets.

The project successfully implements the Medallion Architecture by organizing data into Bronze, Silver, and Gold layers. The Bronze layer preserves raw source data, the Silver layer performs data validation and quality checks, and the Gold layer generates business-ready fact and dimension tables for analytical reporting.

The solution also incorporates enterprise data engineering features such as incremental loading, audit logging, rejected record handling, business rule validation, and Slowly Changing Dimension (SCD) Type 2 implementation for customer data. These features improve data quality, maintain historical information, and support efficient pipeline execution.

Finally, the curated Gold layer data is published to Azure SQL Database and consumed by Power BI dashboards, enabling business users to access accurate, reliable, and analytics-ready data for reporting and decision-making. The implemented solution provides a scalable, maintainable, and reusable data engineering framework that aligns with modern enterprise data platform practices.

11. Unit Testing and Quality Assurance

11.1 Introduction

Ensuring the accuracy, consistency, and reliability of data is a critical requirement in enterprise data engineering solutions. Since the Enterprise Retail Analytics Platform processes data from multiple heterogeneous sources through an automated ETL framework, comprehensive testing was performed to validate each stage of the pipeline. The objective of testing was to verify that the implemented solution correctly ingests, transforms, validates, and publishes data while maintaining data integrity throughout the processing lifecycle.

The testing activities covered Azure Data Factory pipelines, Databricks notebooks, Delta Lake processing, metadata-driven framework execution, incremental loading, Slowly Changing Dimension (SCD) implementation, and data publishing to Azure SQL Database. Both functional validation and data quality verification were performed to ensure that the solution satisfied the project requirements.

11.2 Testing Objectives

The primary objectives of the testing process were:

- Verify successful execution of Azure Data Factory pipelines.
- Validate data ingestion from all configured source systems.
- Ensure correct loading of data into Bronze, Silver, and Gold layers.
- Verify metadata-driven execution for all datasets.
- Validate data cleansing and transformation logic.
- Verify watermark-based incremental loading.
- Validate Slowly Changing Dimension (SCD Type 2) implementation.
- Ensure successful publishing of Gold tables to Azure SQL Database.
- Verify audit logging and rejected record generation.
- Confirm overall data consistency across all processing stages.

11.3 Testing Strategy

The testing process was divided into multiple phases to validate each component independently before validating the complete end-to-end workflow.

Unit Testing

Individual notebooks and reusable framework functions were tested independently to verify their expected behavior. Each notebook was executed using different datasets to ensure that the implemented logic produced the expected output.

Integration Testing

Integration testing verified that Azure Data Factory correctly orchestrated all Databricks notebooks and that data flowed successfully between Raw, Bronze, Silver, Gold, Azure SQL Database, and Power BI.

Data Validation Testing

Data validation ensured that all implemented validation rules correctly identified invalid records, generated rejected datasets, and preserved valid records.

End-to-End Testing

Complete pipeline execution was tested from source ingestion through Azure SQL publishing to confirm that the entire workflow executed successfully without failures.

11.4 Quality Assurance Approach

The project follows a metadata-driven quality assurance framework in which validation rules are maintained centrally within the metadata configuration. During execution, these rules are automatically applied to each dataset without requiring hardcoded validation logic.

The implemented quality assurance process includes:

- Primary key validation
- Mandatory column validation
- Data type validation
- Numeric validation
- Date validation
- Email format validation
- Business rule validation
- Positive value validation
- Duplicate record detection
- Rejected record generation
- Audit logging

This centralized validation framework ensures consistent data quality across all datasets while improving maintainability and scalability.

11.5 Unit Test Cases

The following unit test cases were executed during project implementation.

Test Case ID	Component	Test Description	Expected Result	Status
UT-01	Azure Data Factory	Verify successful execution of Source-to-Raw pipeline	Data copied to Raw layer	Passed
UT-02	Bronze Notebook	Verify Bronze Delta table creation	Bronze table created successfully	Passed
UT-03	Silver Notebook	Verify schema application and validation	Valid records written to Silver	Passed
UT-04	Silver Notebook	Verify duplicate removal	Duplicate records removed	Passed
UT-05	Silver Notebook	Verify mandatory column validation	Invalid records rejected	Passed
UT-06	Silver Notebook	Verify business rule validation	Invalid business records rejected	Passed
UT-07	Incremental Loading	Verify watermark processing	Only new records processed	Passed
UT-08	Gold Notebook	Verify dimension table creation	Dimension tables generated	Passed
UT-09	Gold Notebook	Verify fact table generation	Fact table generated successfully	Passed
UT-10	SCD Type 2	Verify customer history tracking	Historical versions maintained	Passed
UT-11	Azure SQL Publishing	Verify Gold table publishing	Tables available in Azure SQL	Passed
UT-12	Audit Framework	Verify audit log generation	Audit records created	Passed

11.6 Data Quality Validation

Data quality validation was performed during the Silver layer processing to ensure that only high-quality data progressed to the analytical layer.

The validation process included:

- Removal of duplicate records using configured primary keys.
- Validation of mandatory attributes to prevent incomplete records.
- Conversion of data into expected data types.

- Validation of numeric values against configured constraints.
- Standardization of date and timestamp formats.
- Validation of email addresses using regular expressions.
- Verification of business rules defined in the metadata configuration.
- Identification of invalid records and storage within the rejected records repository.

This process ensured that only validated and standardized data was propagated to the Gold layer.

11.7 Incremental Load Validation

Incremental loading was tested for datasets configured with watermark columns.

The testing verified that:

- Previous watermark values were correctly retrieved.
- Only newly inserted or modified records were processed.
- Existing records were not reprocessed.
- Watermark values were successfully updated after completion.

This testing confirmed that incremental processing reduced unnecessary data movement while maintaining data consistency.

11.8 SCD Type 2 Validation

Customer Dimension was tested to verify the implementation of Slowly Changing Dimension (SCD Type 2).

The following scenarios were validated:

- New customer insertion.
- Customer attribute modification.
- Historical record preservation.
- Effective start and end date updates.
- Current record identification using status indicators.

Testing confirmed that historical customer information was accurately preserved without overwriting previous records.

11.9 Audit and Logging Validation

Audit logging was validated throughout the processing pipeline.

The verification included:

- Generation of pipeline execution identifiers.
- Recording of ingestion timestamps.

- Recording of processing timestamps.
- Creation of rejected record indicators.
- Successful generation of audit log entries for each processed dataset.

These validations ensure complete traceability and operational monitoring of the ETL process.

11.10 Test Results Summary

All implemented components were successfully tested and validated. Azure Data Factory pipelines executed without errors, Databricks notebooks processed datasets according to the configured metadata, validation rules correctly identified invalid records, watermark-based incremental loading functioned as expected, SCD Type 2 maintained historical customer information, and Gold tables were successfully published to Azure SQL Database.

The completed testing process confirms that the implemented solution is functionally correct, maintains data quality, and satisfies the business and technical requirements defined for the Enterprise Retail Analytics Platform.

12. Project Snapshots

This chapter presents snapshots of the implemented Azure Data Engineering solution. The screenshots demonstrate the successful configuration and execution of the major project components, including Azure resources, Azure Data Factory pipelines, Azure Databricks notebooks, Delta Lake layers, Azure SQL Database, and Power BI dashboards. These snapshots provide visual evidence of the implemented solution and validate the successful execution of the end-to-end data engineering pipeline.

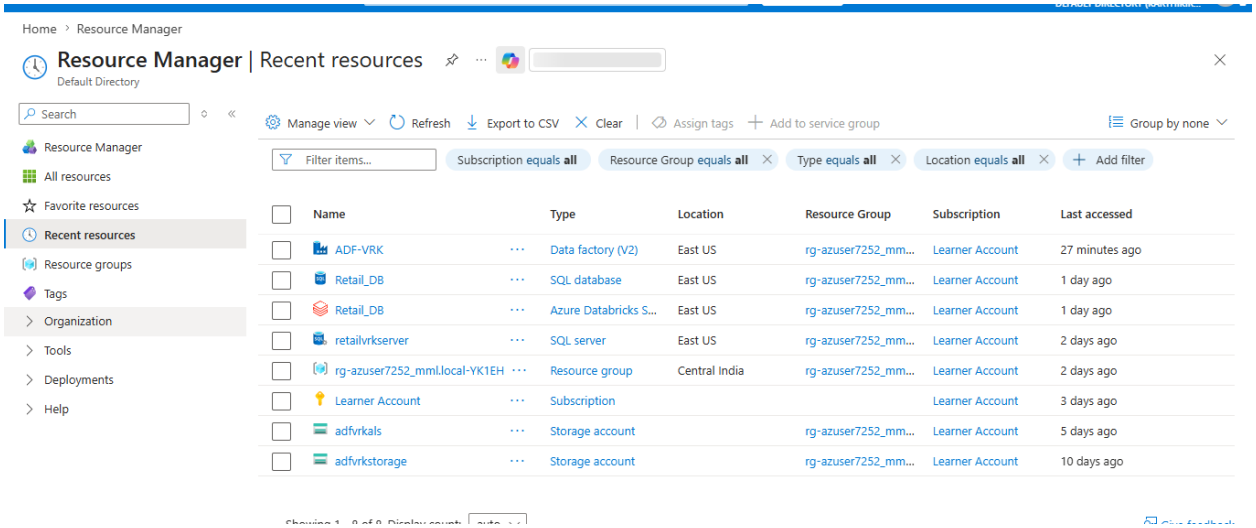


Figure 12.1: Azure Resources

This screenshot is of the Azure resources created for the project. The deployed resources include Azure Data Factory, Azure Data Lake Storage Gen2, Azure Databricks Workspace, Azure SQL Database, and the associated resource group.

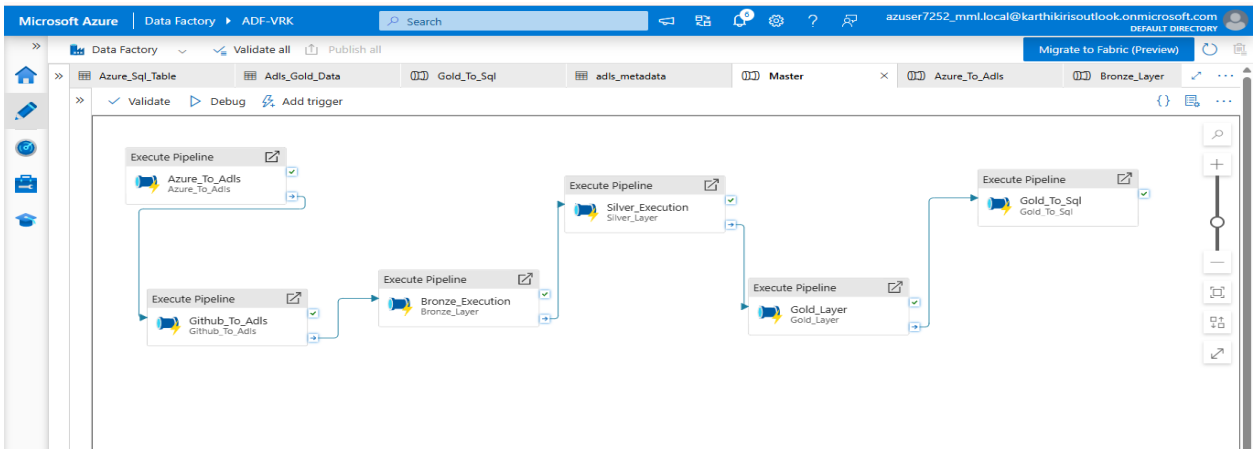


Figure 12.2: Azure Data Factory Pipeline

The Master Pipeline orchestrates the complete ETL workflow by sequentially executing all dependent pipelines from source ingestion through Azure SQL publishing.

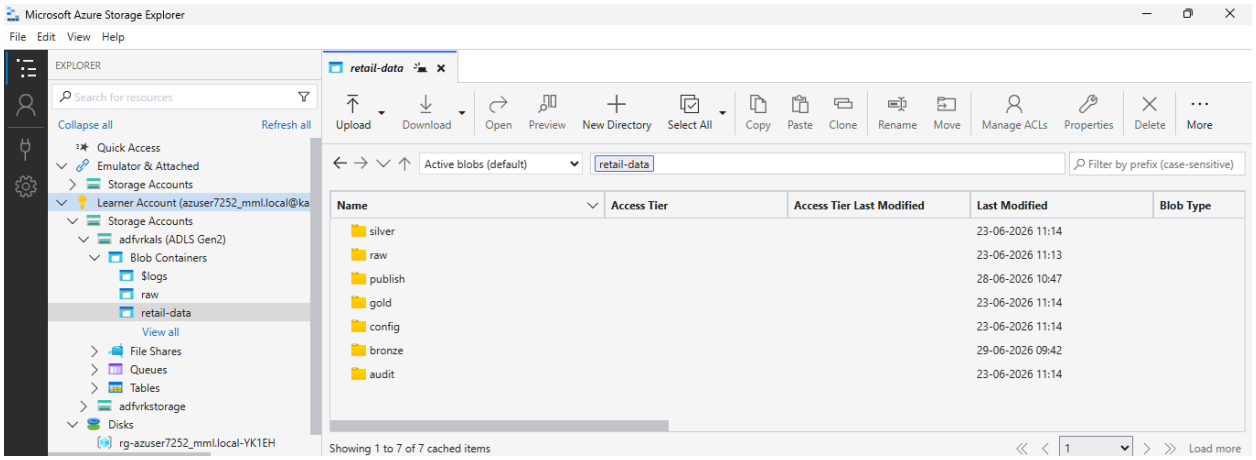


Figure 12.3 ADLS Folder Structure

The above screenshot illustrate the storage organization implemented in Azure Data Lake Storage Gen2. The folder structure follows a logical separation of Raw, Bronze, Silver, Gold, ,Audit ,Publish and Config folders to support data ingestion, processing, and monitoring.

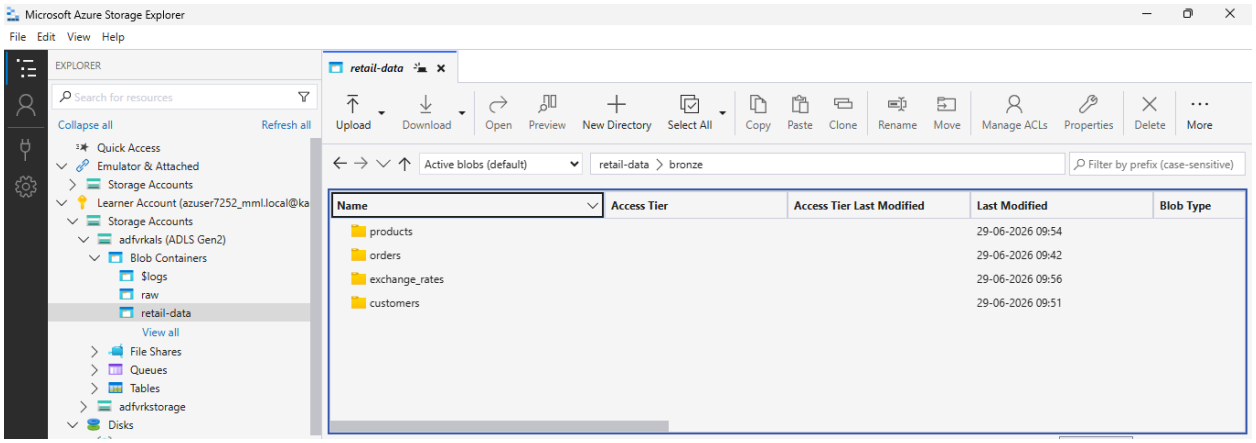


Figure 12.4 Bronze Layer

The above Screenshot shows the successfully created Bronze Delta tables containing raw ingested datasets.

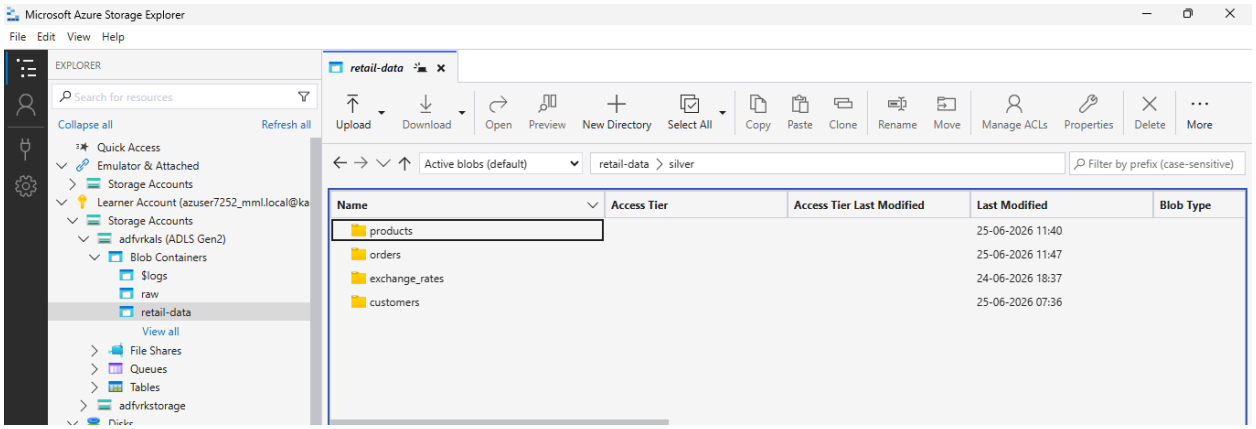


Figure 12.5 Silver Layer

The above Screenshot shows the validated and standardized Silver Delta tables formed after cleaning and validating bronze data.

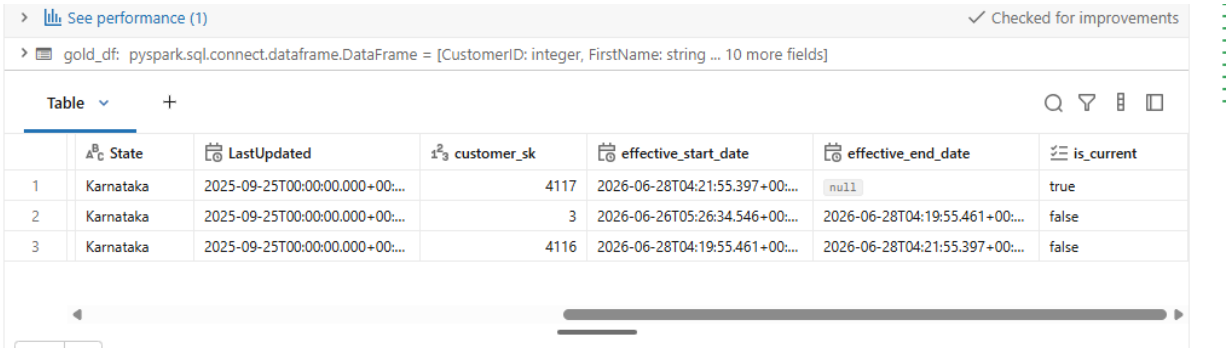


Figure 12.6 SCD 2 Implementation on Customers

This screenshot demonstrates the implementation of **Slowly Changing Dimension (SCD) Type 2** for the Customer Dimension. The framework automatically detects changes in tracked attributes (City and State) and preserves historical information by creating a new version of the record instead of overwriting existing data. Each record is maintained with a surrogate key, effective start date, effective end date, and current status indicator, enabling historical analysis and complete data lineage.

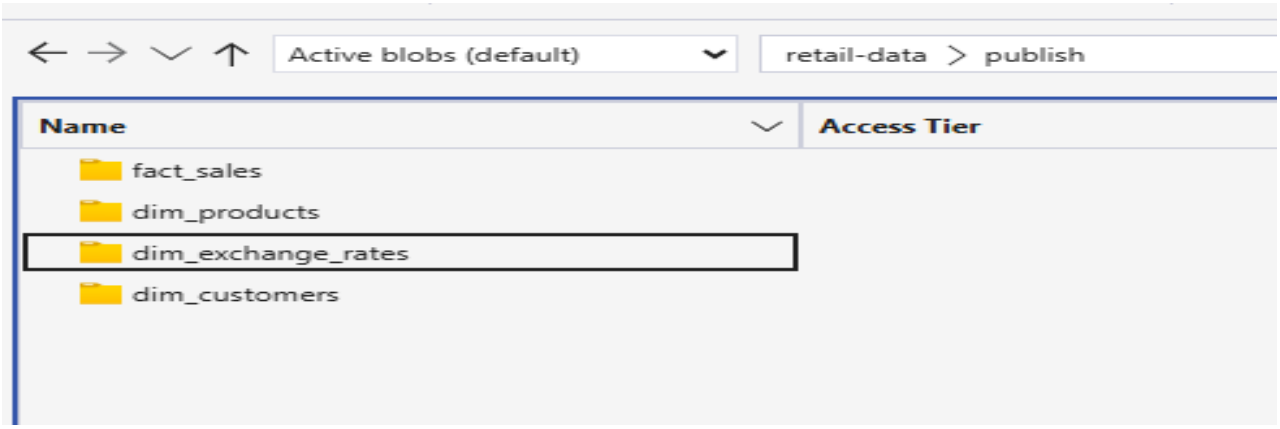
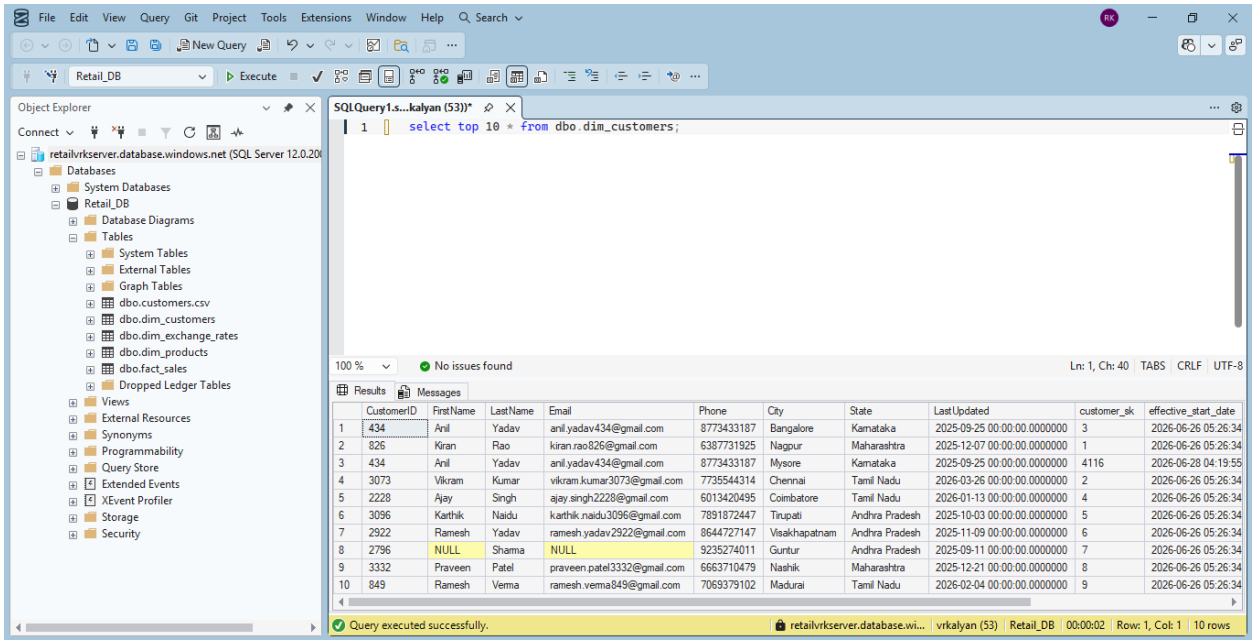


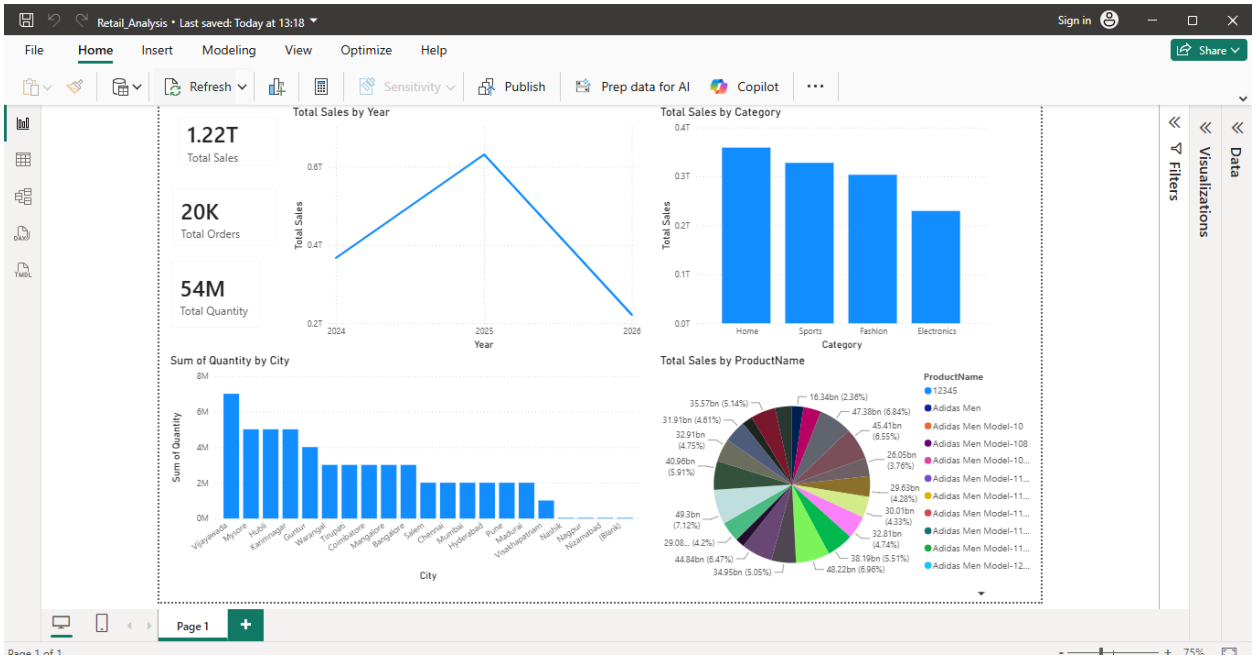
Figure 12.7 Gold Data

This screenshot illustrates the Azure Data Factory pipeline responsible for publishing the curated Gold layer tables to Azure SQL Database. The pipeline transfers the business-ready dimension and fact tables from Azure Data Lake Storage Gen2 into Azure SQL using parameterized Copy Activities.



12.8 Curated Data in Azure SQL

This screenshot shows the successful connection established between Azure Data Factory and Azure SQL Database through a configured Linked Service.This allows curated Gold layer tables to be loaded into the target database for downstream reporting and analytical consumption.



12.9 Retail Sales Analytics Dashboard Using Power BI